TRAIN SIM CLASSIC · VEHICLE WORKFLOW GUIDE

# Part 5

# Lua scripting: custom features on top of the simulation

Scripts cannot rewrite TSC's physics, but they can read and write every control value, move animations, switch nodes, talk to the rest of the consist and show messages. That is enough to build trolley poles, thermal damage, interlocks and safety systems.

## In this part


1. Where scripts live and how they are compiled
2. The callbacks the game calls
3. The API: Call and SysCall
4. The architecture every good vehicle script shares
5. Pattern: the virtual control
6. Worked example: dual trolley pole (tested)
7. Sketch: voltage selector and thermal damage
8. Talking to the rest of the train
9. Debugging
10. Pitfalls


API names come from the community “TS2015 Scripting Reference” supplied with this project (itself unofficial and marked work in progress) and the Kuju sample scripts. The trolley-pole script ships as `examples/lua/TrolleyPole.lua` and passes an offline test ( `tools/lua_mock/run_trolley_test.py` ) against a mock of the API.

# 1 · Where scripts live and how they are compiled

- A blueprint's `ScriptComponent` holds a path to a `.lua` file. The sample electric has **two**: the engine blueprint points at `Sample_EngineScript.lua` and the engine simulation blueprint at `Electric Engine Simulation.lua`. verified Keep your logic in one of them (normally the engine blueprint's) and leave the other empty or absent to avoid two scripts fighting over controls.
- BPE2's Export compiles `.lua` to `.out` in Assets. The game runs the `.out`. A syntax error fails the export, so read the output pane.
- Wagons and tenders can have scripts too (doors, lamps, loads).
- TSC embeds an older Lua. Write conservatively: avoid the `#` length and `%` modulo operators and newer library calls. check the exact version against your build; the example script already follows this rule.

# 2 · The callbacks the game calls

| Function you define | When it runs | Use it for |
|---|---|---|
| `Initialise()` | Once at scenario load, before the route is ready | Variables; `Call("*:BeginUpdate")` |
| `Setup()` | Once when the script is initialised (e.g. when the player clicks the consist) | Player-specific setup |
| `Update(interval)` | Every frame after BeginUpdate; `interval` = seconds since the last frame | All continuous logic |
| `OnControlValueChange(name, index, value)` | When the player moves a cab control | Intercepting and filtering input |
| `OnConsistMessage(id, content, direction)` | A message from another vehicle arrives | Multiple units, wagon doors |
| `OnCameraEnter(cabEnd, carriageCam)` / `OnCameraLeave()` | The camera enters/leaves a cab or passenger view | Cab sounds, occlusion |
| `OnCustomSignalMessage(message)` | A signal or track marker on the route sends a string | Voltage zones, bad-wire markers |

**Once you define OnControlValueChange, you own every control write**

Both Kuju sample scripts contain only one function, and it does exactly this:

```
function OnControlValueChange(name, index, value)
  if Call("*:ControlExists", name, index) then
    Call("*:SetControlValue", name, index, value)
  end
end
```

If you define the callback and forget to write the value back, levers stop working. Note the sample's test: `ControlExists` returns **1 or 0**, and in Lua **0 is true** (only `nil` and `false` are false). The Kuju check therefore always passes. Write `== 1`. **verified**

# 3 · The API: Call and SysCall

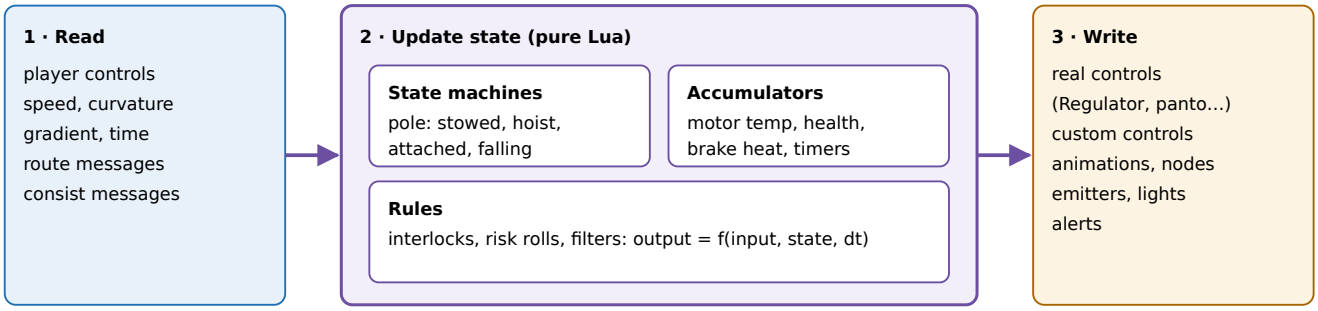
`Call("target:Method", …)` talks to an entity. `"*:"` means “this vehicle and all its children”; `"Main Smoke Stack:"` targets the child entity of that name. `SysCall("Manager:Method", …)` talks to game-wide managers.

| Call | Returns / notes |
|---|---|
| `Call("*:GetControlValue", name, 0)` | Current value of a control |
| `Call("*:SetControlValue", name, 0, v)` | Writes a control (the bus, Part 1) |
| `Call("*:ControlExists", name, 0)` | 1 or 0 (compare with `== 1`) |
| `Call("*:LockControl", name, 0, 1\|0)` | Lock a control against the player (incompletely documented) |
| `Call("*:GetSpeed")` / `GetAcceleration` | m/s and m/s² |
| `Call("*:GetGradient")` | Track grade |
| `Call("*:GetCurvature")` / `GetCurvatureAhead(d)` | 1 / radius in metres (0 = straight) |
| `Call("*:GetTractiveEffort")` | Current TE (fraction of MaxForce per the reference; `GetControlValue("TractiveEffort")` gives klbf) |
| `Call("*:GetIsPlayer")` / `GetIsEngineWithKey` | 1/0: is this the player's train / the driven engine |
| `Call("*:GetSimulationTime")` | Seconds since the scenario started |
| `Call("*:GetTotalMass")` / `GetConsistTotalMass` / `GetConsistLength` | tonnes / tonnes / metres |
| `Call("*:GetCurrentSpeedLimit")`, `GetNextSpeedLimit(dir, dist)` | m/s; next limit returns type, speed, distance |
| `Call("*:GetNextRestrictiveSignal", dir, dist)` | type, state, distance, aspect |
| `Call("*:AddTime", animID, seconds)` / `Call("*:Reset", animID)` | Advance / rewind an animation; AddTime returns non-zero at the end |
| `Call("*:ActivateNode", nodeName, 1\|0)` | Show / hide a geometry node (mesh swaps, lamps) |

| Call | Returns / notes |
|---|---|
| `Call("child:SetText", text, 0\|1)` | Dynamic text on a child |
| `Call("light:Activate", 1\|0)`, `SetRange`, `SetColor(r,g,b)` | Light children |
| `Call("emitter:SetEmitterActive", 1\|0)`, `SetEmitterRate`, `SetEmitterColour` | Smoke, steam, sparks |
| `Call("*:SendConsistMessage", id, content, dir)` | Message along the train |
| `Call("ControlSound:SetParameter", sound, v)` | Drive a sound parameter |
| `SysCall("ScenarioManager:ShowAlertMessageExt", head, msg, secs, 0)` | Pop-up alert (4th argument required, use 0) |
| `SysCall("ScenarioManager:ShowMessage", msg)`, `ShowInfoMessageExt(…)` | Messages / info boxes |
| `SysCall("ScenarioManager:GetTimeOfDay")`, `GetSeason` | Seconds since midnight; 0 spring ... 3 winter |
| `SysCall("ScenarioManager:TriggerScenarioFailure", msg)` | End the scenario (e.g. terminal burnout) |

Calls seen in design snippets but **absent from the reference**: `Call("PlaySound", …)`, `SysCall("ScenarioManager:ShowAlertMessage", …)` (without Ext), `SetTime`. **check** before use. Sounds are normally driven by control values that the sound blueprint listens to, which is how the example script does it.

# 4 · The architecture every good vehicle script shares




Every Update(): read everything once → update state with dt → write everything once. Keep game calls at the edges.

**Figure 5.1 · Read → update → write.** Keeping all `Call`s at the edges makes the middle testable offline, which is exactly how the trolley-pole script was tested without the game.

- Scale everything by `interval` (dt). Frame rates vary; per-frame probabilities (like the spec's `math.random(1,500)`) make behaviour depend on fps.
- Gate expensive or player-only logic with `GetIsPlayer`. AI trains run the same script.
- Do one-time work that needs a spawned vehicle in the first `Update`, not in `Initialise`.

# 5 · Pattern: the virtual control

To put your own logic between the player and the physics, give the player a lever that writes a **custom** control, and let the script write the **real** one:

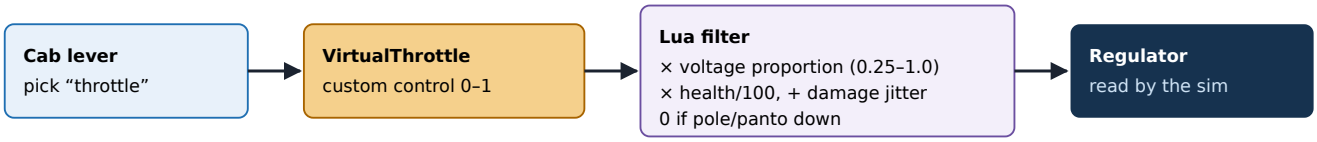



**Figure 5.2 · Virtual control.** The lever's interface element points at `VirtualThrottle`; `Regulator` keeps its name so the simulation still finds it but has no lever of its own.

# 6 · Worked example: dual trolley pole

`examples/lua/TrolleyPole.lua` implements the revised trolley-pole spec. Its state machine per pole:

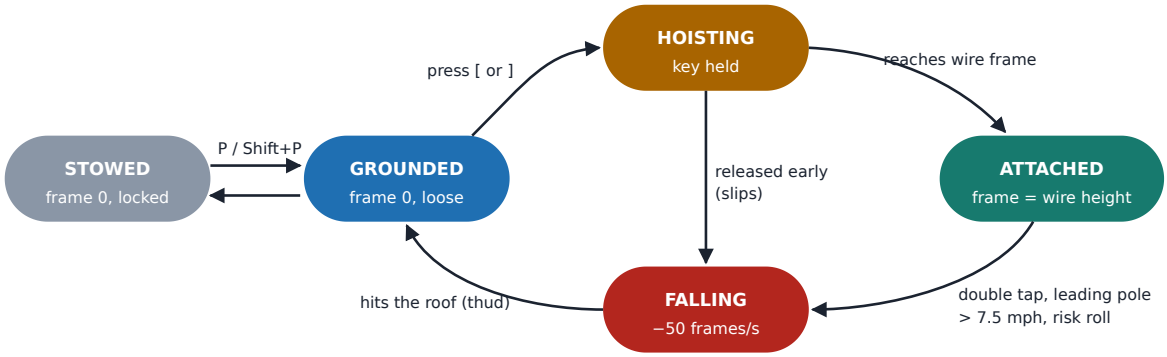



**Figure 5.3 · Pole states from the revision spec.** Power flows only while at least one pole is ATTACHED; the script drives `PantographControl` as the interlock (an assumption **check**: that your electric subsystem gates traction on its pantograph input).

## What the blueprint must provide

| Item | Purpose |
|---|---|
| Controls `PoleFrontKey`, `PoleRearKey` | Held = 1 by the [ and ] keys through the input mapper |
| `PoleFrontStow`, `PoleRearStow`, `PoleHeightMode`, `PoleHeightAdjust` | P / Shift+P, Alt+P, comma/period |
| `PoleFrontFrame`, `PoleRearFrame` | Outputs for cab indicators (0–100) |
| `PoleSpark`, `PoleThud` | One-frame pulses the sound blueprint triggers on |
| AnimSet `poleFront`, `poleRear` | 0–100 frame .ia exported at 100 fps (Part 3 §7) |

| Item | Purpose |
|---|---|
| Nodes `pole_front` / `pole_front_stowed` (and rear) | Mesh swap for the stowed state via `ActivateNode` |

## Core of the script

```
-- Position an exterior animation at an absolute frame with documented calls only
local function showFrame(pole)
  Call("*:Reset", pole.anim)
  if pole.frame > 0 then
    Call("*:AddTime", pole.anim, pole.frame / ANIM_FPS)
  end
  set(pole.out, pole.frame)
end

-- ATTACHED: follow the wire, roll for detachment (scaled by dt, not per frame)
elseif pole.state == SEATED then
  pole.frame = target
  if leading and not held and mph > LEADING_LIMIT_MPH then
    enter(pole, FALLING)
  else
    local risk = detachRisk(pole, leading, held, mph)
    if risk > RISK_THRESHOLD and math.random() < (risk / 500) * RISK_PER_SECOND * 60 * dt then
      enter(pole, FALLING)
    end
  end
elseif pole.state == FALLING then
  pole.frame = pole.frame - FALL_RATE * dt          -- 92 → 0 takes 1.84 s, 65 → 0 takes 1.30 s
```

## Tested behaviour

The offline test drives the script at 60 updates per second against a mock API and checks 14 behaviours: starts stowed, unstow swaps meshes, hoists and seats at preset frame 65, positions the animation at 0.65 s, stays up after release, double-tap de-wire, fall timing, power interlock on and off, the once-per-session reminder (and that it does not repeat), rolling re-wire, leading-pole detachment above 7.5 mph, and Alt+P preset cycling. All pass.

> **Spec numbers worth revisiting**
>
> - The gameplay spec's bad-wire alert fires at **50–70 mph** and faster risk decay starts below **40 mph**, while leading poles detach at **5–10 mph**. For an interurban these may be intended; for street running they are very high. They are constants at the top of the script.
> - The spec's pole range changed from 0.0–1.0 (original) to frames 0–100 (revision). The script follows the revision.
> - `GetCurvature` returns 1/radius in metres; the spec's formula has no unit, so the script uses curvature per km × mph × 0.1. Tune in game.

# 7 · Sketch: voltage selector and thermal damage

The advanced traction and thermal framework is a larger version of the same architecture. Its skeleton:

```
function Update(dt)
  local lineV   = gZoneVoltage or DESIGN_VOLTAGE          -- from OnCustomSignalMessage zone
markers; fallback = design
  local mode    = get("VoltageSelector")                   -- 0 = 600 V parallel, 1 = 1200 V
series
  local motorV  = (mode == 0) and lineV or lineV / 2
  local prop    = 0.5 * (motorV / 600)                      -- 0.25 starved, 0.5 normal, 1.0
overvolt

  -- Phase A: reversible heat; V² rule; fans scale with axle speed
  local heat    = stress() * (motorV / 600) ^ 2
  gMotorTemp    = clamp(gMotorTemp + (heat - cooling(mph)) * dt, 0, 100)
  -- Phase B: irreversible health loss at 100 %, flashover and spin-down stress
  if gMotorTemp >= 100 then gHealth = gHealth - MELT_RATE * dt end
  -- Phase C: virtual throttle
  set("Regulator", get("VirtualThrottle") * prop * 2 * (gHealth / 100) + jitter(gHealth))
end
```

Brake fade follows the same shape (heat ∝ brake × speed, fade multiplier down to 0.2, cooling when released) and writes a scaled brake value through a virtual brake control. Flat spots use a hidden brake control pulsed for one frame per wheel revolution. Everything the framework needs is covered by `GetSpeed`, control reads/writes, `ActivateNode`, emitters and alerts.

# 8 · Talking to the rest of the train

- `Call("*:SendConsistMessage", id, content, direction)` sends a string along the consist; the receiver's `OnConsistMessage(id, content, direction)` fires.
- Forward messages you don't consume, so they reach the far end (the reference's convenience function does exactly this).
- Use your own numeric id range (e.g. 424200+) to avoid clashing with other vehicles' scripts.
- Direction encoding is inconsistent in the reference (0/1 when sending, 1/−1 when receiving). **check** by logging both ends.
- Typical uses: multiple-unit throttle and pantograph, door commands to wagons, tail-lamp control on the last vehicle.

# 9 · Debugging

```
DEBUG = true
function DebugShowAlert(message, timeout)
  if DEBUG then
    SysCall("ScenarioManager:ShowAlertMessageExt", "Debug", message, timeout or 2, 0)
  end
end
```

- `print()` output and script errors appear in **LogMate**. A runtime error usually stops the script for the rest of the session. Check LogMate first when a feature “just stops”.
- Show state on screen with alerts while developing; remove them or gate with `DEBUG` before release.
- Test logic offline: keep game calls at the edges (Figure 5.1) and run the script against a mock, as `tools/lua_mock/run_trolley_test.py` does.

# 10 · Pitfalls

| Pitfall | Fix |
|---|---|
| `if Call("…Exists") then` always true | Compare with `== 1` |
| Defining `OnControlValueChange` without writing values | Pass through everything you don't intercept |
| Per-frame random rolls | Scale probabilities by `dt` |
| Writing a control the player is also moving | Virtual control pattern |
| Two scripts (engine + sim) writing the same control | One owner per control |
| Control name typos | Constants at the top of the script (the toolkit's Lua skeleton generates them from Blender tags) |
| AI trains running player effects | Gate with `GetIsPlayer` |
| Newer Lua syntax ( `#t` , `a % b` ) | Loops with counters, `math.mod` or arithmetic |